

Workable Item Lifecycle

The workable-items **SQLite DB is the single source of truth** (§11.4.93, tracked in git per §11.4.95). Issues / Fixed / the summaries and their HTML / PDF / DOCX siblings are **generated outputs** (§11.4.12 / §11.4.53 / §11.4.65 / §11.4.106). **Never hand-edit a generated doc** — change the DB and regenerate, or the next regeneration silently reverts your edit.

Status – closed set (§ . . / § . . / § . .)

Queued · In progress · Ready for testing · In testing · Reopened · Operator-blocked · terminal closure · Obsolete (→ Fixed.md)

Type – closed set (§ . .)

Bug · Feature · Task . No fourth value. Task is the lowest-stakes default when genuinely ambiguous.

Closure vocabulary is TYPE-AWARE
(§ . .)

Type	Closure Status
Bug	Fixed (→ Fixed.md)
Feature	Implemented (→ Fixed.md)
Task	Completed (→ Fixed.md)

Closing a `Feature` as "Fixed", or a `Task` as "Implemented", is a §11.4.33 violation. All three are semantically equivalent ("closed, positive evidence captured") but the literal must match the Type.

1 1 4 1 0 8

Closure requires captured evidence – always (§ . . . / § . . .)

An item may be closed **only** with rock-solid captured proof (§11.4.5 / §11.4.69 / §11.4.107). **Forbidden as proof:** metadata-only, configuration-only, absence-of-error, and grep-without-runtime.

"Done" means the item's **runtime signature verifies on a clean deployment** (§11.4.108) — source-committed ≠ artifact-contains-it ≠ active-on-clean-target ≠ works-for-the-user. Green at the source layer says nothing about the other three.

If you are unsure **how** to validate something, that is a research trigger, not a licence to accept a weak PASS: do the deep research first (§11.4.123).

Reopening (§ . . . / § . . .)

A reopen is a **demotion from closed** and therefore requires positive evidence captured under the conditions that re-exposed the defect (§11.4.7). Record:

- **By:** `AI` or `User`
- **On:** ISO date
- **Reason:** from the closed vocabulary — `test-failed` · `manual-testing-detected` · `captured-evidence-contradicts` · `end-user-report` · `cycle-re-discovered` · `design-reconsidered`
- **Evidence:** path to the captured artefact

Items with `reopens_count > 0` carry a per-item reopen history doc. A high reopen count is the strongest empirical fragility signal — those cases get the **deepest live-testing scrutiny, and they get it FIRST** (§11.4.189 / §11.4.132).

Operator-blocked is a LAST resort (§ . .)

Only after documenting exhaustion of every self-resolution path (available CLI/API access, subagent delegation, existing tooling, a captured fallback, external research). It must state **WHAT / WHY (each exhausted alternative) / UNBLOCK CONDITION / WHO**. A fake `operator-blocked` is a covenant violation at the planning layer.

Obsolete needs a reason + a triple check (§ . .)

Reasons: `superseded-by-design-change` · `superseded-by-later-mandate` · `feature-removed` · `duplicate-of` · `unsupported-topology` · `not-reproducible` . Removing an existing end-user capability additionally requires **asking the operator first** (§11.4.122) — never a silent removal.

Nothing may sit un-wired forever (§ . .)

Every started effort — research, spike, design doc, half-landed feature — reaches a **terminal state**: fully COMPLETED *and wired* (integrated and used by default, not parked behind a dead flag) **or** explicitly, evidence-backed CLOSED. There is no third "sitting in the backlog un-wired" state.

Per-item testing diary (§ . .)

Append-only, one entry per test event: date/time · tested-by · result · observations · action taken + why. **A PASS entry without an evidence path is rejected by the schema** — that constraint is the anti-bluff mechanism, not paperwork.

Practical rules

- Update Status as work actually progresses — a stale Status is a lie the whole pipeline reads.
- Every status change flows **DB** → **regenerate docs**, in the same commit.
- Fix ⇒ a permanent regression guard (§11.4.135), authored RED on the broken artifact and flipped GREEN on the fixed one (§11.4.115).